

Word2Vec Summary

1. 배경 및 목표

기존의 단어 표현 방식(예: One-Hot Encoding)은 단어 간의 관계를 반영하지 못하고, 고차원 벡터로 표현되는 단점이 있음.

이를 해결하기 위해 Word2Vec은 단어의 의미적 유사성을 벡터 공간에서 보존하는 저차원 연속 벡터 표현을 학습하는 방법을 제안함.

목표

- 단어를 고차원 희소 벡터(Sparse Representation) 대신, 저차원 밀집 벡터(Dense Representation) 로 변환하도록 함.
- 단어 간의 의미적 유사성(Semantic Similarity) 을 벡터 연산을 통해 포착하도록 함.
- 기존의 계산 비용이 큰 방법(예: LSA, NNLM)보다 빠르고 효율적인 단어 임베딩 학습 방법을 개발함.

2. Word2Vec 모델 구조

Word2Vec은 단어의 맥락(Context) 정보를 활용하여 임베딩을 학습하는 방법을 제시함.

핵심 아이디어는 다음 두 가지 방식으로 구현됨.

(1) CBOW (Continuous Bag of Words) 모델

- 주어진 문맥(Context) 단어들을 이용해 중심(Target) 단어를 예측하는 방식임.
- 예제:
 - 문장: "The cat sits on the mat"
 - 문맥 단어: ["The", "cat", "on", "the", "mat"]
 - 중심 단어 예측: "sits"
- CBOW의 특징
 - 주변 단어를 보고 중심 단어를 예측함.
 - 작은 데이터셋에서 상대적으로 성능이 좋음.
 - 학습 속도가 빠름.

(2) Skip-gram 모델

- 중심 단어로부터 주변(Context) 단어를 예측하는 방식임.
- 예제:
 - 중심 단어: "sits"
 - 예측해야 하는 문맥 단어: ["The", "cat", "on", "the", "mat"]
- Skip-gram의 특징
 - 드문 단어까지 잘 학습할 수 있음(희소한 데이터에서 성능 우수).
 - CBOW보다 학습 속도는 느리지만 일반적으로 성능이 더 좋음.

Word2Vec은 대량의 텍스트 데이터에서 단어 임베딩을 학습하는 모델임. 하지만 전체 코퍼스를 대상으로 확률 분포를 직접 계산하는 것은 **연산량이 너무 크기 때문에 비효율적**임. 이를 해결하기 위해 **효율적인 학습 기법**들이 사용됨.

1. Word2Vec 학습 과정

Word2Vec의 핵심 아이디어는 맥락(Context) 정보를 활용하여 단어 벡터를 학습하는 것. 두 가지 방식이 있음.

(1) CBOW (Continuous Bag of Words) 학습 과정

- 입력: 주변 단어(Context)
- 출력: 중심 단어(Target)

예제 문장

| "The cat sits on the mat"

➡ 중심 단어: "sits"

➡ 주변 단어: ["The", "cat", "on", "the", "mat"]

CBOW는 주변 단어를 입력으로 받아 중심 단어를 예측하는 방식임.

1. 주변 단어들을 **One-Hot Encoding** 형태로 변환함.
2. 입력층(Embedding Layer) 을 통해 **밀집 벡터(Dense Vector)**로 변환됨.
3. 벡터들을 평균 내어 하나의 벡터로 만듦.

4. 이 벡터를 이용하여 **Softmax 분류기**를 통과하여 **중심 단어를 예측**함.
5. 예측된 단어와 실제 단어의 차이를 **오차 함수(Loss Function)** 를 이용해 계산하고, 이를 바탕으로 **역전파(Backpropagation)** 및 **경사 하강법(Gradient Descent)**으로 가중치를 업데이트함.

(2) Skip-gram 학습 과정

- 입력: 중심 단어(Target)
- 출력: 주변 단어(Context)

Skip-gram은 CBOW와 반대로, 중심 단어 하나를 입력으로 받아 **주변 단어들을 예측**하는 방식임.

예제 문장

| "The cat sits on the mat"

➡ 중심 단어: "sits"

➡ 예측해야 할 주변 단어: ["The", "cat", "on", "the", "mat"]

Skip-gram의 학습 과정:

1. 중심 단어를 **One-Hot Encoding** 형태로 변환함.
2. **입력층(Embedding Layer)** 을 통과하면서 밀집 벡터(Dense Vector)로 변환됨.
3. 이 벡터를 기반으로 **각 주변 단어를 예측**하는 Softmax 분류기를 거침.
4. 예측된 단어와 실제 주변 단어의 차이를 계산하여 **오차를 최소화**하도록 학습함.

CBOW보다 느리지만, 희소한 데이터에서도 성능이 더 좋음.

2. 최적화 기법

CBOW와 Skip-gram 모델은 **전체 코퍼스의 단어 확률을 직접 계산**해야 하므로, **계산량이 매우 큼**. 이를 해결하기 위해 여러 가지 최적화 기법이 도입됨.

(1) 네거티브 샘플링 (Negative Sampling)

💡 "전체 단어 분포를 계산하는 대신, 일부 단어만 샘플링하여 학습"

기존 문제점: Softmax의 높은 연산 비용

- Word2Vec에서 단어를 예측하는 과정에서는 **Softmax 함수를 사용하여 확률 분포를 계산**해야 함.
- 하지만 코퍼스에 있는 단어가 수백만 개라면, 모든 단어의 확률을 계산하는 것은 **연산량이 너무 많아 비효율적**임.

해결책: 네거티브 샘플링

- 목표 단어(Positive Sample)와 함께 무작위로 샘플링한 단어들(Negative Sample)을 학습함.
- 즉, 일부 단어만 비교하여 학습하는 방식임.

◆ 예제

- "sits" 라는 단어를 중심 단어로 예측하는 경우,
- "on", "the" 같은 실제 문맥 단어(Positive Sample)와 함께,
- "banana", "airplane", "computer" 같은 무작위 단어(Negative Sample)를 샘플링하여 학습함.
- **목표:** 긍정적인 예제(문맥 내 단어)와 부정적인 예제(무관한 단어)를 구별하는 방식으로 학습함.

✅ 효과:

- 모든 단어 확률을 계산할 필요 없이 일부 샘플만 학습하므로 연산량이 크게 감소함.
- 모델 성능을 유지하면서도 학습 속도가 빨라짐.

(2) Subsampling (자주 등장하는 단어 제거)

💡 "빈번한 단어를 확률적으로 제거하여 더 중요한 단어 학습을 강화"

기존 문제점

- "the", "is", "and" 같은 단어들은 매우 자주 등장하지만 의미적인 정보가 적음.
- 이런 단어들이 너무 자주 학습되면, 상대적으로 중요한 단어들이 제대로 학습되지 않음.

해결책: 서브샘플링

- 특정 빈도수 이상 등장하는 단어는 일정 확률로 제거함.
- 단어 w 가 제거될 확률은 다음과 같음:

$$P(w) = 1 - \sqrt{\frac{t}{f(w)}}$$

- $f(w)$ = 단어 w 의 빈도수
- t = 특정 임계값 (보통 10^{-5} 정도)

◆ 예제

- "the" 같은 단어는 등장 빈도가 높기 때문에 **제거 확률이 높아짐**.
- "elephant", "algorithm" 같은 단어는 등장 빈도가 낮아 **제거되지 않음**.

✓ 효과:

- 학습 속도를 높이고, 더 중요한 단어들에 집중할 수 있도록 함.
- 드문 단어들의 임베딩 품질을 개선할 수 있음.

(3) Hierarchical Softmax

💡 "단어를 트리 구조로 표현하여 Softmax 계산을 최적화"

기존 문제점

- Skip-gram 모델에서 중심 단어로부터 주변 단어를 예측할 때,
 - Softmax를 계산하려면 **모든 단어의 확률을 계산**해야 함.
 - 단어가 많을수록 연산량이 너무 많아짐.

해결책: 계층적 소프트맥스

- 단어를 **Huffman 트리** 형태로 표현하고,
- Softmax 대신 **이진 트리 탐색(Binary Tree Search)** 방식으로 확률을 계산함.

◆ 예제

- "sits" 단어를 예측하는 경우,
- 단어 전체를 탐색하는 대신, **트리의 루트에서 리프까지 탐색**하며 예측 확률을 계산함.

✓ 효과:

- Softmax를 직접 계산하는 것보다 연산량이 **$O(\log N)$** 으로 감소함.

- 단어 개수가 많아도 빠르게 학습할 수 있음.
-

4. Word2Vec의 결과 및 활용

Word2Vec은 단어의 의미적 관계를 벡터 공간에 잘 반영한다는 것을 수많은 실험을 통해 보여줌.

(1) 단어 벡터 연산

- 단어 벡터를 수학적으로 연산할 수 있음.
- 예제:
 - **"king" - "man" + "woman" $\approx$ "queen"**
 - 즉, 벡터 공간에서 의미적인 관계를 유지함.

(2) 유사도 계산

- 단어 벡터 간의 코사인 유사도(Cosine Similarity)를 이용해 관련 단어를 검색할 수 있음.
- 예: **"Paris"** 와 유사한 단어를 찾으면 **"France"** 가 나옴.

(3) 실제 활용 예시

- 문서 분류, 감성 분석, 추천 시스템 등에 적용할 수 있음.
 - 구글, 페이스북, 아마존 등 여러 기업에서 활용됨.
-

5. 결론 및 한계

(1) 기여

- 단어 임베딩을 효율적으로 학습할 수 있는 CBOW, Skip-gram 모델을 제안함.
- 기존 방법보다 빠르고 계산 비용이 낮음.
- 의미적 유사성을 반영하는 단어 벡터를 생성할 수 있음.

(2) 한계점

- 문맥을 고려하지 않음 (동일 단어라도 문맥에 따라 의미가 다를 수 있음).
- OOV(Out-of-Vocabulary) 문제 발생 → 훈련 데이터에 없는 단어를 다룰 수 없음.
- 문장 단위의 의미 표현이 어려움.

➡ 이후 연구에서는 Word2Vec의 한계를 보완하는 방법으로 FastText(문자 단위 임베딩), ELMo(문맥 기반), BERT(Transformer 기반) 등이 제안됨.